Bounty Hacker

Plataforma: TryHackMe

Dificuldade: Fácil

Data de Resolução: 12/04/2026

Sumário

1. Visão Geral 3
2. Reconhecimento 4
3. Exploração 5
4. Escalada de Privilégios 6
5. Conclusão 7
6. Referências 7

1. Visão Geral

O desafio "Bounty Hacker" consiste na exploração de um servidor web e FTP mal configurados, permitindo o acesso anônimo a arquivos sensíveis. A superfície de ataque inicial explorada focou no vazamento de uma lista de senhas e identificação de um usuário válido. O caminho até o comprometimento total envolveu um ataque de força bruta contra o serviço SSH, seguido pela exploração de privilégios excessivos concedidos ao binário tar via sudo para escalada local.

---

2. Reconhecimento

A fase de enumeração iniciou-se com uma varredura completa de portas no alvo. Como a varredura padrão estava demasiadamente lenta, o comando foi otimizado utilizando a flag --min-rate 5000 para forçar o envio rápido de pacotes e contornar a lentidão, identificando rapidamente as portas abertas:

```bash
nmap -p- --min-rate 5000 -T4 <TARGET_IP>
```

Justificativa: O comando mapeia todo o espectro de portas (1 a 65535) utilizando a flag `-p-`. O parâmetro `--min-rate 5000` acelera a execução forçando o Nmap a enviar não menos que 5.000 pacotes por segundo, enquanto `-T4` define um perfil de tempo agressivo.

Com as portas 21, 22 e 80 identificadas, uma segunda varredura focada foi executada para extrair as versões exatas dos serviços em execução, permitindo uma análise de vulnerabilidades mais precisa:

```bash
nmap -sV -p 21,22,80 <TARGET_IP>
```

Justificativa: A flag `-sV` instrui o Nmap a interagir ativamente com as portas especificadas (`-p 21,22,80`) para extrair os banners e determinar as versões exatas dos serviços e softwares nelas alocados.

Output truncado:

```plaintext
```

```
PORT     STATE   SERVICE VERSION
21/tcp   open    ftp     vsftpd 3.0.5
22/tcp   open    ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.13
(Ubuntu Linux; protocol 2.0)
80/tcp   open    http    Apache httpd 2.4.41 ((Ubuntu))
```

---

3. Exploração

O serviço FTP (porta 21) rodando o vsftpd 3.0.5 foi o vetor de entrada selecionado. A versão em si não possuía falhas críticas públicas, mas configurações inseguras são comuns neste protocolo. Uma tentativa de login utilizando as credenciais padrão de acesso anônimo (usuário: anonymous, senha em branco) foi realizada com sucesso:

```bash
ftp <TARGET_IP>
```

Justificativa: O comando inicia a conexão de rede com o serviço FTP no alvo. O uso da credencial genérica "anonymous" testa falhas básicas de controle de acesso ao sistema.

Dentro do servidor FTP, os arquivos foram listados e copiados para a máquina local para análise minuciosa:

```bash
ls -la
binary
mget *
exit
```

Justificativa: `ls -la` lista todo o conteúdo do diretório (incluindo arquivos ocultos). `binary` previne a corrupção de arquivos alterando o modo de transferência. `mget *` instrui o servidor a enviar todos os arquivos listados para a máquina atacante simultaneamente, e `exit` encerra a conexão.

A inspeção dos arquivos baixados revelou informações críticas para o prosseguimento do ataque. O arquivo task.txt continha anotações que indicavam um nome de usuário válido no sistema (lin). Simultaneamente, o arquivo locks.txt funcionava como um dicionário de senhas personalizadas:

```bash
cat task.txt
cat locks.txt
```

Justificativa: O comando `cat` (concatenate) lê e exibe o conteúdo dos arquivos de texto diretamente no terminal local, permitindo a extração do possível nome de usuário e dicionário de senhas.

Com um usuário válido (lin) e uma wordlist (locks.txt), um ataque de força bruta online foi direcionado ao serviço SSH (porta 22) utilizando a ferramenta Hydra:

```bash
hydra -l lin -P locks.txt ssh://<TARGET_IP>
```

Justificativa: O Hydra automatiza as tentativas de login na rede. A flag `-l lin` fixa o teste para o usuário específico encontrado na etapa anterior, enquanto `-P locks.txt` fornece a wordlist local para testes de tentativa e erro contra o serviço SSH alvo.

Output truncado:

```plaintext
[22][ssh] host: <TARGET_IP>   login: lin   password:
<PASSWORD>
```

A credencial obtida permitiu o acesso remoto seguro ao servidor. A primeira flag foi imediatamente localizada e validada no diretório inicial do usuário:

```bash
ssh -p 22 lin@<TARGET_IP>
cat user.txt
```

Justificativa: A conexão remota é estabelecida usando o comando nativo do protocolo SSH com a porta e as credenciais previamente quebradas. Em seguida, `cat` é usado para leitura e confirmação da captura da flag inicial do desafio.

[Output: `<FLAG>`]

4. Escalada de Privilégios

Após estabelecer o acesso inicial, a enumeração local focou na verificação de permissões administrativas configuradas incorretamente. O comando sudo -l revelou que o usuário lin poderia executar o binário /bin/tar como superusuário sem restrições:

```
Bash
sudo -l
```

Justificativa: A flag `-l` enumera as permissões ativas do arquivo *sudoers*, listando exatamente quais programas o usuário atual tem autorização para executar com privilégios de `root`.

O binário tar possui a funcionalidade de "checkpoints", que permite a execução de comandos arbitrários no sistema a cada número pré-definido de arquivos processados. Utilizando este recurso em conjunto com as permissões do sudo, um payload foi construído para forçar o tar a abrir um shell de comando (/bin/sh), herdando os privilégios máximos:

```
Bash
sudo tar -cf /dev/null /dev/null --checkpoint=1
--checkpoint-action=exec=/bin/sh
```

Justificativa: Invoca o binário como superusuário (`sudo tar`) e cria um arquivo descartável (`-cf /dev/null`). Os parâmetros `--checkpoint=1` e `--checkpoint-action=exec=/bin/sh` abusam de uma funcionalidade legítima do programa para interceptar o fluxo de execução e abrir o binário `/bin/sh` (interpretador de comandos do sistema operacional) mantendo os direitos de administrador de quem iniciou o processo original.

O sucesso da escalada foi confirmado, garantindo controle total do sistema para a coleta da flag final:

```
Bash
whoami
```

Justificativa: Um comando básico de verificação de identidade que retorna o nome do usuário atualmente autenticado no shell em uso, atestando a eficácia da escalada de privilégios.

[Output: `root`]

```bash
cd /root
cat root.txt
```

Justificativa: `cd /root` (change directory) força a navegação para o diretório raiz exclusivo do superusuário, que antes estava inacessível. O uso do `cat` expõe e coleta a flag definitiva da máquina.

[Output: `<FLAG>`]

---

5. Conclusão

O comprometimento da máquina Bounty Hacker originou-se do mapeamento de um servidor FTP com login anônimo habilitado, resultando no vazamento de um nome de usuário e uma lista de senhas. Estas informações viabilizaram um ataque de força bruta bem-sucedido contra o serviço SSH. Por fim, a máquina foi totalmente dominada explorando uma falha de configuração de privilégios no arquivo sudoers, abusando do recurso de checkpoints do binário tar para obter uma shell interativa como root.

---

6. Referências

- **GTFOBins - Tar:** https://gtfobins.github.io/gtfobins/tar/#sudo
- **Hydra (Kali Tools Documentation):** https://www.kali.org/tools/hydra/
- **Guia de Pentesting FTP (HackTricks):** https://book.hacktricks.xyz/network-services-pentesting/pentesting-ftp
- **RFC 959 - File Transfer Protocol (Documentação Oficial):** https://datatracker.ietf.org/doc/html/rfc959